



January 2023

Acerca de arc42

arc42, La plantilla de documentación para arquitectura de sistemas y de software.

Por Dr. Gernot Starke, Dr. Peter Hruschka y otros contribuyentes.

Revisión de la plantilla: 7.0 ES (basada en asciidoc), Enero 2017

© Reconocemos que este documento utiliza material de la plantilla de arquitectura arc42, <https://www.arc42.org>. Creada por Dr. Peter Hruschka y Dr. Gernot Starke.

Introducción y Metas

El proyecto Savio Mobile es una adaptación de la página web Savio (Sistema de Aprendizaje Virtual Interactivo) de la Universidad Tecnológica de Bolívar (en adelante UTB), diseñada para que estudiantes y docentes puedan interactuar en el marco de sus cursos a través de múltiples recursos y herramientas (lecturas, libros, tareas, exámenes, entre otros).

La plataforma web Savio es, en esencia, una versión personalizada de Moodle, un sistema de gestión del aprendizaje (LMS, por sus siglas en inglés) de código abierto ampliamente utilizado en el ámbito académico para la administración de cursos, calificaciones y actividades educativas.

El software en desarrollo tiene como propósito principal llevar todas las funcionalidades de la plataforma web a una aplicación móvil, con el fin de ofrecer una experiencia más práctica, accesible y cómoda. De esta manera, los estudiantes podrán conocer las clases programadas, gestionar sus actividades académicas, recibir notificaciones sobre tareas próximas a vencer y acceder a todos los servicios que Savio ya proporciona, pero ahora desde la palma de su mano.

Vista de Requerimientos

Como requerimientos funcionales, este proyecto propone lo siguiente:

- La aplicación móvil debe ser capaz de soportar una alta afluencia de usuarios simultáneamente.
- La aplicación móvil debe permitir el inicio de sesión con las credenciales institucionales de la UTB y conectarse directamente con la plataforma Savio.
- La aplicación móvil debe incorporar la identidad visual de la UTB (colores, logotipo y estilo gráfico).
- La aplicación móvil debe ser compatible con sistemas operativos Android e iOS, manteniendo una experiencia de usuario consistente.
- La aplicación móvil debe enviar notificaciones automáticas sobre vencimientos de actividades, publicación de calificaciones, anuncios y cambios en el horario.

Metas de Calidad

Como equipo de desarrollo, esperamos cumplir con las siguientes metas de calidad de nuestro proyecto:

- Integrar características en la aplicación que reflejen la identidad institucional de la UTB y respondan a las particularidades de la comunidad académica.

- Ampliar las posibilidades de acceso a los recursos educativos a través de la aplicación.
- Desarrollar una experiencia de usuario enriquecida y alineada con las necesidades de estudiantes y docentes de la UTB.
- Evolucionar la aplicación hacia un entorno digital que supere una adaptación básica y se ajuste a las necesidades específicas de la universidad.
- Garantizar que la aplicación sea fácil de usar, con una interfaz clara y coherente con la identidad institucional de la UTB.
- Mantener una documentación básica pero suficiente para que otros estudiantes o desarrolladores puedan comprender y continuar con el proyecto en el futuro.
- Entregar una versión de la aplicación que sea útil y práctica para los estudiantes, aunque no cuente con todas las características avanzadas de un producto comercial.
- Utilizar de manera eficiente el uso de herramientas gratuitas y de código abierto, aprovechándolas al máximo para garantizar un buen desarrollo y despliegue de la aplicación.

Partes interesadas (Stakeholders)

Rol/Nombre	Contacto	Expectativas
Willy J. Laza	wlaza@utb.edu.co	<i>El stakeholder espera que el proyecto parta de la base de Moodle y logre una conexión directa con Savio, lo que implica gestionar los permisos y accesos necesarios. Además, confía en que se documenten claramente los requerimientos para garantizar dicha integración y que se aproveche el ambiente de pruebas Saviotest para validar las funcionalidades con usuarios reales.</i>

Rol/Nombre	Contacto	Expectativas
Jairo E. Serrano	jserrano@utb.edu.co	<i>El stakeholder espera que el proyecto aproveche la base de Moodle para establecer una conexión directa y fluida con Savio, gestionando los permisos y accesos necesarios para garantizar una integración efectiva en producción. Asegurando así que el proyecto esté listo para su implementación y despliegue exitoso en producción.</i>

Restricciones de la Arquitectura

El diseño de la arquitectura del proyecto *Savio Mobile* está condicionado por una serie de restricciones que deben ser consideradas para garantizar la viabilidad técnica y la alineación con los deseos de nuestros stakeholders. Estas restricciones son las siguientes:

- **Dependencia de Moodle:** La aplicación móvil debe integrarse directamente con Moodle (base de Savio), lo que implica ajustarse a su modelo de datos, API y actualizaciones oficiales. No se podrán realizar cambios estructurales en la base de datos central de Moodle, por lo que cualquier adaptación deberá hacerse mediante extensiones, conectores o servicios externos.
- **Compatibilidad y plataformas:** La aplicación móvil debe ser compatible con Android e iOS, utilizando un framework que permita mantener una sola base de código (por ejemplo, Flutter o React Native). La interfaz debe conservar la identidad visual de la UTB y ser consistente con las pautas de usabilidad establecidas.
- **Seguridad y autenticación:** El inicio de sesión debe realizarse exclusivamente mediante las credenciales institucionales de la UTB, integrándose con los protocolos de autenticación ya definidos en Savio. Todos los datos personales y académicos deben transmitirse mediante conexiones seguras (HTTPS/TLS) y almacenarse de acuerdo con las políticas de privacidad de la universidad.
- **Infraestructura y despliegue:** La aplicación dependerá de la infraestructura ya disponible de la UTB y de la administración de Moodle/Savio; por lo tanto, no se contempla el uso de servidores externos comerciales para la

base de datos. Las pruebas de integración deberán realizarse en el entorno de pruebas *Saviotest* antes de cualquier despliegue en producción.

- **Limitaciones de recursos:** El proyecto utilizará prioritariamente herramientas gratuitas y de código abierto, lo que restringe el uso de servicios de pago o dependencias comerciales.

Alcance y Contexto del Sistema

Contexto de Negocio

El sistema consiste en una aplicación móvil multiplataforma (Android/iOS) que extiende la funcionalidad de la plataforma educativa institucional basada en *Moodle*. El propósito de la aplicación es brindar a estudiantes y docentes una experiencia más práctica y adaptada a dispositivos móviles, manteniendo la integridad de los datos y la lógica de negocio centralizada en los servicios de la UTB.

Alcance:

- Acceso autenticado mediante credenciales institucionales.
- Visualización y gestión de cursos inscritos.
- Acceso a recursos educativos (archivos, enlaces, videos, foros).
- Envío de tareas y consulta de calificaciones.
- Notificaciones móviles sobre eventos académicos relevantes.

Fuera de alcance:

- Administración avanzada de Moodle (gestión de usuarios, configuración global).
- Desarrollo de nuevos plugins para Moodle.
- Cambios en la infraestructura principal de Moodle.

Actores principales:

- Estudiantes: consumen recursos, participan en actividades, realizan entregas de tareas.

Contexto Técnico

La aplicación móvil se sitúa como cliente dentro del ecosistema de servicios de la institución, actuando como una capa de presentación sobre Moodle.

Sistemas externos relevantes:

- **Moodle LMS:** sistema base que gestiona cursos, usuarios y actividades. La app móvil consume sus APIs REST para obtener y enviar información.

- **Sistema de autenticación institucional:** utilizado para validar credenciales y mantener consistencia con el portal web.
- **Servicios de contenido externo:** repositorios de videos, enlaces o documentos que Moodle referencia en los cursos.
- **API dedicada a la aplicacion (REST):** El sistema se conectara a la API para el interambio de datos.

Interacciones técnicas:

- La aplicación móvil se comunica con Moodle a través de la *Moodle Web Service API (REST)*.
- El flujo de inicio de sesión se delega al sistema de autenticación institucional.
- Para ciertos recursos, la aplicación ofrece almacenamiento en caché local para acceso sin conexión.

Estrategia de solución

Objetivo de Calidad	Escenario	Enfoque de Solución
Identidad institucional	La aplicación debe reflejar la identidad visual y funcional de la institución	Personalización de la interfaz con logos, colores institucionales y navegación adaptada a la estructura de cursos existente
Accesibilidad a recursos educativos	Los estudiantes deben poder acceder a todos los recursos de los cursos desde la app móvil, incluso offline	Uso de almacenamiento en caché de recursos, integración con APIs de Moodle y enlaces a contenido externo
Experiencia de usuario enriquecida	Los usuarios deben navegar fácilmente y completar tareas sin frustración	Aplicación nativa móvil, interfaces adaptativas, navegación intuitiva y notificaciones relevantes
Adaptación institucional avanzada	La app debe cubrir necesidades específicas de la institución más allá de la adaptación básica de Moodle	Extensión de funcionalidades mediante configuraciones de cursos y foros, integración de herramientas libres ya existentes
Usabilidad y coherencia	La interfaz debe ser clara y consistente en todas las pantallas y módulos	Uso de patrones de diseño consistentes, guías de estilo y pruebas de usabilidad con estudiantes/docentes
Documentación mínima suficiente	Otros estudiantes y desarrolladores deben poder entender y continuar el proyecto	Documentación básica de arquitectura, README detallado, comentarios claros en el código
Eficiencia y herramientas open-source	La aplicación debe aprovechar al máximo las herramientas gratuitas y de código abierto	Selección de bibliotecas open-source maduras, APIs de Moodle, optimización de recursos para no depender de soluciones propietarias

Vista de Bloques

Descripción general

El sistema está conformado por un conjunto de bloques interrelacionados que integran tanto la capa móvil (Ionic + Cordova) como el backend Moodle. Estos

bloques se organizan de forma modular para facilitar la mantenibilidad, la reutilización y la escalabilidad del sistema.

Bloques principales

- **Bloque de Interfaz de Usuario (UI):** Contiene las pantallas y componentes visuales desarrollados con Ionic. Gestiona la navegación, interacción con el usuario y visualización de contenidos de Moodle.
- **Bloque de Lógica de Aplicación:** Implementa la lógica de negocio del cliente móvil, incluyendo autenticación, sincronización de datos, manejo de sesiones, y lógica de caché offline.
- **Bloque de Comunicación API:** Gestiona la comunicación con el servidor Moodle a través de servicios REST. Incluye los controladores que encapsulan las peticiones HTTP y la serialización de respuestas JSON.
- **Bloque de Persistencia Local:** Administra el almacenamiento interno del dispositivo (IndexedDB o almacenamiento nativo), permitiendo el uso offline y la sincronización posterior de datos.
- **Bloque de Integración Cordova:** Reúne los plugins nativos que permiten el acceso a funcionalidades del dispositivo como cámara, notificaciones, almacenamiento, conexión de red y archivos.
- **Bloque de Configuración y Dependencias:** Contiene los archivos `package.json`, `config.xml` y `capacitor.config.ts`, que definen versiones, dependencias y configuraciones específicas de compilación.

Interrelaciones entre bloques

- La **Interfaz de Usuario** interactúa con la **Lógica de Aplicación** para ejecutar las acciones del usuario.
- La **Lógica de Aplicación** se comunica con la **API de Moodle** mediante el **Bloque de Comunicación**.
- Los datos obtenidos del servidor se almacenan o consultan desde el **Bloque de Persistencia Local**.
- El **Bloque de Integración Cordova** actúa de intermediario entre el entorno híbrido y las capacidades nativas del dispositivo.

Ventajas del diseño modular

- Facilita la sustitución o actualización de componentes sin afectar el resto del sistema.
- Permite una clara separación de responsabilidades, lo que mejora la mantenibilidad del código.

- Favorece la reutilización de componentes en futuras versiones o proyectos derivados.

Vista de Ejecución

Diferencia de entornos

El sistema de Moodle utiliza como base Linux, lo que genera un sesgo de compatibilidad frente a usuarios que desarrollan en Windows. Esta situación obligó al equipo a recurrir a Windows Subsystem for Linux (WSL) como alternativa, dado que MacOS y Linux nativos no presentaban estas limitaciones. Aun así, se identificaron múltiples incompatibilidades, diferencias de rendimiento y fallos en librerías clave durante el desarrollo y la generación de la aplicación móvil (APK).

- **Caracteres inválidos en Windows:** Algunos documentos y scripts de configuración utilizaban caracteres válidos en Linux (como el ; en separaciones), pero no aceptados en Windows. Incluso bajo WSL estos errores persistían, ya que Windows continuaba imponiendo restricciones de sistema de archivos.
- **Debugging lento en Windows:** El proceso de depuración era significativamente más largo. En Linux o MacOS el tiempo promedio no superaba los 2 minutos, mientras que en Windows con WSL el mismo procedimiento tardaba entre 5 y 8 minutos, obligando al equipo a cerrar cualquier aplicación en segundo plano para liberar recursos.
- **Compilación con Android Studio:** En entornos Windows, la creación de la APK era aún más costosa. Además del tiempo de depuración, era necesario generar el paquete de la aplicación y levantar el simulador de Android, duplicando el tiempo de espera en comparación con entornos Linux nativos.
- **Problemas de compatibilidad con librerías:** Dependencias como *Ionic* mostraban incompatibilidades en Windows debido a que varias versiones se encontraban descontinuadas, aunque seguían siendo requeridas por la última versión estable de Moodle. Esto obligaba a recurrir a configuraciones manuales y versiones específicas de paquetes.
- **Fallas en comandos principales:** El comando estándar `npm run start` fallaba en entornos Windows. Como alternativa, algunos desarrolladores debieron emplear un comando extendido:

```
npx cross-env NODE_OPTIONS=--max-old-space-size=8192
NODE_ENV=production ionic build --prod
```

Este comando evitaba bloqueos de memoria y permitía construir la aplicación en Windows, aunque con un costo adicional en rendimiento.

Vista de Ejecución

La vista de ejecución describe cómo los distintos componentes del sistema Moodle Phone interactúan en tiempo de ejecución, qué procesos se activan, cómo se comunican entre sí y cómo se coordinan las tareas entre el cliente móvil, el backend de Moodle y los servicios externos.

Componentes en ejecución

- **Cliente móvil (Ionic/Cordova):** ejecuta la interfaz principal, maneja la navegación, almacenamiento local y sincronización de datos. Utiliza el motor de renderizado web y plugins nativos.
- **Servidor Moodle:** provee las API REST para autenticación, gestión de cursos, usuarios y envío de calificaciones.
- **Servicios complementarios:** pueden incluir almacenamiento de medios, servicios de notificaciones push y servicios de analítica de uso.

Flujo de ejecución típico

1. El usuario inicia sesión desde la aplicación móvil, la cual valida credenciales contra el servidor Moodle.
2. El cliente consulta los cursos disponibles mediante servicios REST.
3. Los datos se almacenan temporalmente en caché local, lo que permite funcionamiento offline.
4. Al reconectarse, los cambios se sincronizan con el servidor de manera controlada.

Consideraciones de rendimiento

- La comunicación entre el cliente y Moodle debe optimizarse mediante peticiones asincrónicas y paginación.
- Los procesos intensivos, como descarga de archivos o sincronización de notas, deben ejecutarse en segundo plano.
- El rendimiento depende tanto del hardware móvil como de la latencia del servidor.

Vista de Despliegue

La vista de despliegue muestra cómo el sistema puede instalarse y ejecutarse en distintos entornos, incluyendo configuraciones locales, entornos de desarrollo y de producción. En este caso, el objetivo es garantizar la compatibilidad entre el entorno de desarrollo (Ionic + Cordova) y el entorno del servidor Moodle.

Ejecución en entornos locales

En un entorno local, el sistema puede desplegarse parcialmente para desarrollo y pruebas:

- **Se puede desplegar en local:**
 - Generación del servidor de modo desarrollo mediante `npm run dev`.
 - Pruebas unitarias y de compilación del APK usando Android Studio o CLI en sistemas Linux o WSL.
 - Ejecución del cliente web desde navegadores o emuladores.
- **No se puede desplegar en local:**
 - Construcción estable de la APK directamente en Windows sin WSL.
 - Ejecución de ciertas librerías dependientes del kernel Linux, como módulos nativos de Node.
 - Instalación de la aplicación móvil oficial sin certificado ni identidad válida.

Requisitos adicionales en Windows

- Uso de **WSL (Windows Subsystem for Linux)** para disponer de herramientas compatibles con el entorno de compilación.
- Configuración de `ANDROID_SDK_ROOT` y `JAVA_HOME` para permitir la construcción del proyecto.
- Instalación de librerías de soporte como Gradle, Cordova CLI y Ionic CLI.

Escenarios de compatibilidad

- **Escenario 1:** Si el repositorio principal de Moodle se actualiza, el sistema debe seguir funcionando o documentar incompatibilidades.
 - En la práctica, los archivos que suelen cambiar son `package.json` y `package-lock.json`.
 - Las dependencias deben validarse antes de actualización para evitar rupturas.
- **Escenario 2:** Si un comando no existe en Windows, debe proveerse alternativa o guía de instalación mediante WSL.
- **Escenario 3:** El tiempo de compilación no debe superar un umbral definido en entornos Linux nativos.

Conceptos Transversales (Cross-cutting)

- **Seguridad:** Manejo seguro de tokens de sesión y almacenamiento local cifrado.
- **Escalabilidad:** Posibilidad de ampliar el número de usuarios conectados mediante balanceo de carga en el backend Moodle.
- **Mantenibilidad:** Separación entre la capa de presentación (Ionic) y la lógica de negocio (API Moodle).
- **Portabilidad:** Ejecución multiplataforma en Android, iOS y navegadores.

Requerimientos de Calidad

- **Rendimiento:** Respuesta menor a 2 segundos en la carga de cursos y recursos.
- **Confiabilidad:** Persistencia de datos locales durante desconexiones.
- **Usabilidad:** Interfaz intuitiva, consistente con la versión web de Moodle.
- **Compatibilidad:** Soporte para Android 8.0 o superior y servidores Moodle 4.0 en adelante.

Árbol de Calidad

Categoría	Atributos
Eficiencia	Rendimiento, consumo de memoria, optimización de peticiones
Confiabilidad	Tolerancia a fallos, sincronización segura, respaldo
Usabilidad	Navegación fluida, retroalimentación visual, accesibilidad
Mantenibilidad	Modularidad, claridad del código, documentación técnica
Portabilidad	Compatibilidad Android/iOS, soporte WSL

Riesgos y Deuda Técnica

- **Dependencia de versiones externas:** Las actualizaciones de Cordova o Moodle pueden requerir ajustes manuales.
- **Errores de compilación cruzada:** Diferencias entre entornos Windows y Linux pueden afectar la generación de APK.
- **Gestión de dependencias:** Incompatibilidades en módulos NPM al actualizar versiones mayores.
- **Riesgo de obsolescencia:** Plugins nativos sin mantenimiento pueden dejar de ser compatibles con nuevas versiones de Android.

- **Deuda técnica:** Falta de automatización en pruebas de integración y ausencia de pipeline continuo.

Glosario

Término	Definición
<i>API</i>	(Application Programming Interface) Conjunto de reglas y definiciones que permiten que el frontend y el backend se comuniquen entre sí mediante solicitudes y respuestas, generalmente usando JSON.
<i>Savio</i>	<i>Savio (Sistema de Aprendizaje Virtual Interactivo) es la plataforma web que la UTB utiliza para darle a sus estudiantes y profesores una manera fácil y rápida de interactuar entre sí. Esta es una modificación del código de Moodle, adaptado a la identidad gráfica de la UTB, usando sus símbolos y colores.</i>
<i>LMS</i>	<i>Sistema de Gestión del Aprendizaje (Siglas en inglés de Learning Management System)</i>
<i>Moodle</i>	<i>Moodle es un sistema de gestión de aprendizaje de código abierto, ampliamente utilizado a nivel mundial en instituciones educativas y organizaciones para apoyar procesos de formación virtual y mixta. Esta plataforma permite la creación y administración de cursos en línea, facilitando la interacción entre docentes y estudiantes mediante actividades como foros, cuestionarios, tareas, evaluaciones y la disponibilidad de diversos recursos académicos.</i>
<i>Stakeholder</i>	<i>Un stakeholder es cualquier persona, grupo u organización que tiene interés, influencia o se ve afectado de alguna manera por un proyecto, una empresa o una iniciativa. En el contexto de proyectos, los stakeholders pueden incluir desde los usuarios finales (como estudiantes y docentes en una aplicación educativa), hasta los patrocinadores, directivos, equipo de desarrollo, e incluso entidades externas que dependen o se ven impactadas por el resultado del proyecto.</i>
<i>Saviotest</i>	<i>Saviotest es la plataforma de pruebas de Savio, y es el principal medio usado para realizar las respectivas pruebas del proyecto aquí documentado.</i>

Término	Definición
<i>APK</i>	<i>(Android Package Kit) Es el formato de archivo utilizado para distribuir e instalar aplicaciones en el sistema operativo Android. En el contexto de este proyecto, se genera a partir de la compilación del código fuente para probar la aplicación en dispositivos móviles.</i>
<i>WSL</i>	<i>(Windows Subsystem for Linux) Es una característica de Windows que permite ejecutar un entorno Linux completo de manera nativa dentro del sistema operativo, facilitando la compatibilidad con librerías y comandos propios de Linux que no funcionan en Windows de forma directa.</i>
<i>Gradle</i>	<i>Herramienta de automatización de construcción utilizada principalmente en proyectos Android. Permite compilar el código, gestionar dependencias y generar paquetes listos para instalación, como los archivos APK.</i>
<i>SDK</i>	<i>(Software Development Kit) Conjunto de herramientas de desarrollo que incluyen compiladores, depuradores y bibliotecas necesarias para crear aplicaciones. En este caso, el SDK de Android es esencial para construir y probar la aplicación en dispositivos móviles.</i>
<i>Repositorio</i>	<i>Espacio centralizado, generalmente en plataformas como GitHub o GitLab, donde se almacena, organiza y versiona el código fuente del proyecto. Su actualización puede generar cambios en dependencias críticas como package.json.</i>
<i>Dependencia</i>	<i>Componente externo (librería, framework o paquete) que el proyecto requiere para ejecutarse o compilarse. Las dependencias se definen en archivos de configuración como package.json y deben instalarse antes de la ejecución.</i>